



11. Customizing Neural Networks Models

"Custom Models, Smarter Outcomes" — A deep dive into Keras

Previous Section:

 [10. Deploying Neural Network Models](#)

Contents:

[1. Three APIs for Building Models](#)

[1.1. Sequential Model](#)

[1.2. Functional API: Thinking in Computational Graphs](#)

[1.3. Model Subclassing](#)

[1.4. Combining Everything Together](#)

[2. Different Ways to Build Keras Models](#)

[2.1. Data Preparation \(Understanding the Problem First\)](#)

[2.2. Ticket Dataset \(Multi-Input / Multi-Task Learning\)](#)

[2.3. Sequential Model \(Linear Stack of Layers\)](#)

[2.4. Functional Model \(Graph-Based Architecture\)](#)

[2.5. Siamese Network \(Learning Similarity\)](#)

[2.4. Subclassing Model \(Full Control\)](#)

[Summary](#)

[References](#)

So far, we have explored many aspects of Deep Learning: from understanding neural network architectures, to evaluating models, improving performance,

and even deploying them into real-world systems. But once we move beyond beginner examples, we quickly realize something important: real-world problems are rarely simple.

In practice, a model may need to process multiple inputs at the same time, produce several outputs, or contain different branches specialized for different types of data. The training process itself may also require custom metrics, specialized loss functions, or callbacks tailored to the problem. Because of this, modern Deep Learning frameworks are designed not only for simplicity, but also for flexibility. This is where Keras becomes extremely powerful.

Keras allows us to gradually move from simple high-level APIs toward lower-level customization while still keeping the code readable and manageable. The important skill is not simply knowing how to build a model, but understanding *when* the built-in APIs are enough, and *when* we need to customize the architecture or training process ourselves.

If we rank the different approaches in Keras by complexity and flexibility, we can roughly think about them like this:

- The simplest level is using `Sequential + compile + fit` for straightforward linear architectures.
- A more flexible level is the Functional API, which allows multiple branches, inputs, and outputs.
- A deeper level of customization comes from subclassing `keras.Model` and manually defining the forward computation.
- The most advanced level involves customizing the training process itself using custom `train_step` methods or even fully manual training loops.

As we move downward through these levels, flexibility increases significantly. However, the risk of bugs, debugging difficulty, and implementation complexity also increase.

So in this chapter, we are not simply learning “how to use Keras”. We are learning how neural network systems are structured, customized, and controlled depending on the complexity of the problem itself.

1. Three APIs for Building Models

Keras provides three primary ways to build neural network models: Sequential API; Functional API; Model Subclassing.

Each approach represents a different level of flexibility and abstraction. None of them are universally “better” than the others. Instead, each one fits different types of problems.

An important thing to understand is that, in Keras, both layers and models behave like callable objects. A layer can be applied directly onto tensors, and even a model itself can be reused as part of another larger model. Because of this, these APIs are not isolated systems. They can be combined together inside the same project.

1.1. Sequential Model

The Sequential model is the simplest and most beginner-friendly way to build a neural network.

A Sequential model is essentially a linear stack of layers arranged from input to output. Data flows through the network step-by-step in a straight pipeline.

This makes Sequential models highly suitable for problems such as:

- Binary classification
- Multi-class classification
- Simple regression tasks
- Basic feedforward neural networks

The biggest advantage of the Sequential API is clarity. The architecture is easy to write, easy to read, and easy to debug. Because of this, Sequential models are commonly used as baselines before experimenting with more advanced architectures.

However, the simplicity of Sequential models also creates limitations. They are not designed for architectures with: Multiple inputs; Multiple outputs; Branching structures; Shared layers; Complex graph connections

In other words, Sequential should only be used when the architecture is truly linear.

There are two common ways to build a Sequential model:

- Passing a list of layers directly into the constructor
- Adding layers incrementally using the `add()` method

Another important concept is that layers only create their weights once the input shape becomes known. This is why specifying input dimensions is important during model construction.

After building a model, we often use `model.summary()` to inspect the architecture. This helps us verify: Output shapes; Number of parameters; Layer connectivity. Naming layers is also extremely useful because it makes summaries easier to read and debugging much more manageable.

Parameter count matters more than many beginners expect. A model with too many parameters can easily become slow to train, memory expensive, highly prone to overfitting.

For example, suppose a Dense layer receives an input vector with 10 features and contains 32 neurons. The parameter count becomes 10×32 weights, 32 bias values

⇒ So the total number of trainable parameters is: $10 \times 32 + 32 = 352$

Even small architectural changes can dramatically increase parameter counts, which is why inspecting model summaries becomes an essential habit in Deep Learning development.

1.2. Functional API: Thinking in Computational Graphs

While Sequential models think in terms of linear pipelines, the Functional API thinks in terms of computational graphs.

The process usually begins with `keras.Input()`, which creates a symbolic tensor representing the input of the model. Layers are then called on tensors to produce new tensors, gradually constructing the graph step-by-step. Finally, the entire graph is wrapped using `keras.Model(inputs, outputs)`.

Unlike Sequential models, the Functional API explicitly tracks how tensors flow between layers. Keras therefore understands the entire graph structure, including nodes, connections, and dependencies. This makes the Functional API ideal for architectures involving: Multiple branches; Multiple inputs; Multiple outputs; Shared layers; Complex feature fusion

For example, imagine a customer support system that receives A ticket title, a detailed description, or a set of tags.

- Each input may require a different processing branch. Later, these branches can be merged using operations such as concatenation. The final model may even produce multiple outputs simultaneously, such as: Priority prediction or Department assignment
- Each output can also use its own loss function and evaluation metrics independently.

One of the most powerful ideas in the Functional API is layer sharing. The same layer object can be called multiple times on different tensors. When this happens, all calls share the exact same weights.

This is extremely useful for architectures such as shared encoders, siamese networks, similarity learning systems.

It is important to distinguish between reusing the same layer object and creating two separate layers with identical configurations. These are not the same thing. Shared layers reuse parameters, while separate layers learn independently.

Another major advantage of the Functional API is introspection. Because Keras stores the graph structure automatically, we can inspect `model.layers`, visualize architectures, extract intermediate activations, and build intermediate models for debugging and analysis

This becomes incredibly useful for: Feature visualization; Model debugging; Activation analysis; Understanding hidden representations

And this is one of the biggest advantages the Functional API has over pure model subclassing.

1.3. Model Subclassing

Model subclassing is the most flexible and advanced way to build models in Keras.

Instead of describing the architecture declaratively, we create a custom Python class that inherits from `keras.Model`.

Inside the constructor `__init__`, we define the internal layers of the model. Then, inside the `call()` method, we explicitly define how data flows through the architecture.

This approach allows us to write highly dynamic behavior using normal Python logic, including: Conditional execution; Loops; Dynamic routing; Custom control flow

Because of this flexibility, subclassing is commonly used for: Research models; Experimental architectures; Reinforcement Learning systems; Dynamic neural networks

However, this flexibility comes with trade-offs. Compared to the Functional API, subclassed models provide less automatic graph information. Some visualization, inspection, and serialization features also become more difficult.

In other words, subclassing gives maximum freedom, but also maximum responsibility.

1.4. Combining Everything Together

In real-world Deep Learning projects, these APIs are often mixed together rather than used independently.

A small model can become a reusable block inside a larger architecture. For example:

- A custom layer can be used inside a Sequential model
- A Functional model can become part of a subclassed model
- A subclassed block can be inserted into a Functional graph

The important mindset is modular thinking. Instead of building one giant monolithic model, we build smaller components with clear responsibilities, then combine them together into larger systems.

2. Different Ways to Build Keras Models

We will reuse a **synthetic ticket dataset** and gradually explore three core Keras paradigms:

- Sequential API → simplest stack of layers
- Functional API → multi-input / multi-output systems
- Subclassing → full control over model logic

2.1. Data Preparation (Understanding the Problem First)

Before building any model, we simulate structured data. Machine Learning models do not learn from theory—they learn from patterns in data.

Classification Dataset

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# We simulate a simple classification task:
# Each sample has 20 numerical features
# Goal: predict one of 3 classes

num_samples = 1500
num_features = 20
num_classes = 3

# Input data: random Gaussian features
# Think of this as raw "observations"
x = np.random.normal(size=(num_samples, num_features)).astype("float32")

# Hidden "true relationship" (like a real-world unknown function)
true_w = np.random.normal(size=(num_features, num_classes)).astype("float32")

# We generate logits using a linear transformation + noise
# This simulates a realistic imperfect dataset
logits = x @ true_w + 0.25 * np.random.normal(size=(num_samples, num_classes))
```

```
# Convert logits → class labels
# This becomes our supervised learning target
y = np.argmax(logits, axis=1).astype("int32")

# Train / validation split
x_train, x_val = x[:1200], x[1200:]
y_train, y_val = y[:1200], y[1200:]

print("x_train.shape:", x_train.shape)
print("y_train.shape:", y_train.shape)
print("x_val.shape:", x_val.shape)
print("y_val.shape:", y_val.shape)
```

We created a dataset where the inputs are random but the labels follow hidden structure. This mimics real-world ML problems.

2.2. Ticket Dataset (Multi-Input / Multi-Task Learning)

Real systems are not single-input with single-output. They often take multiple sources of information or predict multiple outputs


```

ticket_samples = 1200

# Three independent feature groups
# Each represents a different "view" of a ticket

title_data = np.random.randint(0, 2, size=(ticket_samples,
100)).astype("float32")
body_data = np.random.randint(0, 2, size=(ticket_samples, 1
000)).astype("float32")
tags_data = np.random.randint(0, 2, size=(ticket_samples, 1
2)).astype("float32")

print("title_data.shape:", title_data.shape)
print("body_data.shape:", body_data.shape)
print("tags_data.shape:", tags_data.shape)

```

```

title_data.shape: (1200, 100)
body_data.shape: (1200, 1000)
tags_data.shape: (1200, 12)

```

Priority Target (Regression Problem)

We simulate priority as a weighted combination of features.

```

priority_targets = (0.4 * title_data[:, :10].mean(axis=1)
+ 0.4 * body_data[:, :30].mean(axis=1)
+ 0.2 * tags_data[:, :4].mean(axis=1)
+ 0.05 * np.random.normal(size=(ticket_samples,)))

priority_targets = np.clip(priority_targets, 0, 1).astype(
    "float32").reshape(-1, 1)
print(priority_targets[:10])

```

```

[[0.6026952 ]
 [0.39159304]
 [0.55048907]
 [0.5610698 ]
 [0.42549857]
 [0.41000727]
 [0.39608875]
 [0.5027417 ]
 [0.65764564]
 [0.3927231 ]]

```

Interpretation:

- Title contributes moderately
- Body contributes strongly
- Tags contribute weakly
- Noise simulates real-world uncertainty

Department Target (Classification Problem)

```

department_scores = np.stack([title_data[:, :25].mean(axis=
1),
                                body_data[:, :250].mean(axis=
1),
                                body_data[:, 250:500].mean(ax
is=1),
                                tags_data[:, :4].mean(axis=
1)], axis=1)

# Choose highest scoring department
department_targets = np.argmax(department_scores, axis=1).a
stype("int32")
print(department_targets[:20])

```

```
[0 1 0 0 0 1 3 3 3 3 3 0 0 3 1 0 3 3 1 0]
```

Train / Validation Split

```

split = 900

ticket_train_inputs = {"title": title_data[:split],
                       "body": body_data[:split],
                       "tags": tags_data[:split],}

ticket_val_inputs = {"title": title_data[split:],
                     "body": body_data[split:],
                     "tags": tags_data[split:],}

ticket_train_targets = {"priority": priority_targets[:split],
                        "department": department_targets[:split],}

ticket_val_targets = {"priority": priority_targets[split:],
                      "department": department_targets[split:],}

print("Keys of ticket_train_inputs:", ticket_train_inputs.keys())
print("Keys of ticket_train_targets:", ticket_train_targets.keys())
print("Keys of ticket_val_inputs:", ticket_val_inputs.keys())
print("Keys of ticket_val_targets:", ticket_val_targets.keys())

```

```

Keys of ticket_train_inputs: dict_keys(['title', 'body', 'tags'])
Keys of ticket_train_targets: dict_keys(['priority', 'department'])
Keys of ticket_val_inputs: dict_keys(['title', 'body', 'tags'])
Keys of ticket_val_targets: dict_keys(['priority', 'department'])

```

2.3. Sequential Model (Linear Stack of Layers)

Sequential models are simple, fast to build, and suitable for single-input tasks

```
def build_sequential_model():

    # A Sequential model is just a pipeline: Input → Hidden
    # Layers → Output
    inputs = keras.Input(shape=(num_features,), name="features")

    hidden_layer1 = layers.Dense(64, activation="relu")(inputs) # Hidden representation learning
    hidden_layer2 = layers.Dense(32, activation="relu")(hidden_layer1) # Non-linear feature extraction

    # Output layer converts features → probabilities
    outputs = layers.Dense(num_classes, activation="softmax")(hidden_layer2)

    model = keras.Model(inputs=inputs, outputs=outputs, name="sequential_classifier")

    return model
```

Training the model

```
seq_model = build_sequential_model()

seq_model.compile(optimizer=keras.optimizers.Adam(1e-3),
                  loss="sparse_categorical_crossentropy", # Sparse categorical crossentropy because labels are integers
                  metrics=["accuracy"])

history_seq = seq_model.fit(x_train, y_train, validation_data=(x_val, y_val),
                            epochs=5, batch_size=32)
```

2.4. Functional Model (Graph-Based Architecture)

Functional API allows multiple inputs, multiple outputs, shared representations, and complex architectures

```
def build_functional_model():

    # Each input represents a different information source
    title_input = keras.Input(shape=(100,), name="title")
    body_input = keras.Input(shape=(1000,), name="body")
    tags_input = keras.Input(shape=(12,), name="tags")

    ...

    title_input = keras.Input(shape=(100,), name="title")
    -> This creates a symbolic KerasTensor with shape (batch_size, 100). It's a placeholder, not actual data.

    When `title_features = layers.Dense(64, activation="relu", name="title_features")(title_input)` is called:
    Keras uses the symbolic `title_input` (shape: (None, 100)) to infer the input shape for the `Dense` layer.
    The `Dense` layer then automatically creates its weights, for example, a weight matrix of shape (100, 64) for this transformation.

    ...

    # Each branch learns its own representation: (100,) -> (64,)
    title_features = layers.Dense(64, activation="relu", name="title_features")(title_input)
    body_features = layers.Dense(64, activation="relu", name="body_features")(body_input)

    # Merge all information sources into one representation
    # Since title_features: (64,); body_features: (64,); tags_input: (12,)
    # Concatenation: 64 + 64 + 12 = 140
    x = layers.Concatenate(name="concat_features")([title_features,
                                                    body_features, tags_input])
    # Print the shape
```

```

print("x.shape:", x.shape)

# Shared representation layer
shared = layers.Dense(64, activation="relu", name="shared_features")(x)

# Output 1: regression (priority score)
priority_output = layers.Dense(1, activation="sigmoid", name="priority")(shared)

# Output 2: classification (department)
department_output = layers.Dense(4, activation="softmax", name="department")(shared)

return keras.Model(inputs=[title_input, body_input, tags_input],
                    outputs=[priority_output, department_output], name="ticket_model")

# A testing run to show the output of the print method
build_functional_model()

```

x.shape: (None, 140)

Training with Multiple Losses

```

functional_model = build_functional_model()
# functional_model.summary()

functional_model.compile(optimizer="rmsprop",
                        # Each output has its own loss function
                        loss={"priority": "mse", "department": "sparse_categorical_crossentropy"},

```



```

        # Metrics per task
        metrics={"priority": ["mae"],
                 "department": ["accuracy"]}

history_ticket = functional_model.fit(ticket_train_inputs,
                                     ticket_train_targets,
                                     validation_data=(ticket_val_inputs,
                                                    ticket_val_targets),
                                     epochs=3, batch_size=32)

```

Extract Intermediate Features

```

# This model exposes internal representation learning
# Useful for visualization or transfer learning

feature_extractor = keras.Model(inputs=functional_model.inputs,
                                outputs=functional_model.get_layer("shared_features").output,
                                name="feature_extractor")

sample_features = feature_extractor.predict([k: v[:3] for k, v
                                             in ticket_val_inputs.items()])

print(sample_features.shape)

```

(3, 64)

2.5. Siamese Network (Learning Similarity)

Instead of classification, we learn: "Are two inputs similar?"

```
def build_siamese_model():  
  
    input_a = keras.Input(shape=(10,), name="item_a")  
    input_b = keras.Input(shape=(10,), name="item_b")  
  
    # Shared encoder = same weights applied to both inputs  
    shared_encoder = layers.Dense(16, activation="relu", name="shared_encoder")  
  
    encoded_a = shared_encoder(input_a)  
    encoded_b = shared_encoder(input_b)  
  
    print("encoded_a.shape:", encoded_a.shape)  
    print("encoded_b.shape:", encoded_b.shape)  
  
    # Cosine similarity measures angle between vectors  
    similarity = layers.Dot(axes=1, normalize=True, name="cosine_similarity")([encoded_a, encoded_b])  
  
    return keras.Model([input_a, input_b], similarity), shared_encoder  
  
siamese_model, _ = build_siamese_model()  
# siamese_model.summary()
```

```
encoded_a.shape: (None, 16)  
encoded_b.shape: (None, 16)
```

2.4. Subclassing Model (Full Control)

Subclassing allows custom forward pass, dynamic logic, and research-level flexibility

```

class MLPClassifier(keras.Model):

    def __init__(self, num_classes=3):
        super().__init__(name="subclassed_mlp")

        # Explicit layer definitions
        self.dense1 = layers.Dense(64, activation="relu")
        self.dropout = layers.Dropout(0.3)
        self.dense2 = layers.Dense(32, activation="relu")
        self.out = layers.Dense(num_classes, activation="softmax")

    def call(self, inputs, training=False):
        # Forward pass is fully controlled here
        x = self.dense1(inputs)
        # Dropout behaves differently during training vs inference
        x = self.dropout(x, training=training)
        x = self.dense2(x)

        return self.out(x)

```

Training

```

sub_model = MLPClassifier(num_classes=num_classes)

# Build model (required for summary)
_ = sub_model(tf.zeros((1, num_features)))

sub_model.compile(optimizer=keras.optimizers.Adam(1e-3),
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

history_sub = sub_model.fit(x_train, y_train,
                           validation_data=(x_val, y_val),
                           epochs=5, batch_size=32)

```

Key Takeaway

- **Sequential** → simplest pipeline
- **Functional** → flexible graph with multiple inputs/outputs
- **Subclassing** → full control over learning logic

These are not just APIs. They are **three different ways of thinking about model design**

Summary

In this section, we moved beyond the idea of simply using pre-built models and instead explored how neural networks can be **customized to match the structure of real problems**.

We started with the **Sequential API**, which shows how a model can be built as a simple stack of layers. Then we extended this idea using the **Functional API**, which allows us to design more realistic systems with **multiple inputs, shared representations, and multiple outputs**, just like real-world applications such as ticket routing.

Finally, we explored **Model Subclassing**, where we gain full control over the forward pass and model behavior, making it possible to implement highly flexible or research-oriented architectures.

Across all three approaches, the key idea remains the same: **Machine Learning (Deep Learning) models are not fixed templates—they are design tools**.

We are taught that Keras is not just about convenience, but about **adaptability**. Instead of forcing our problem into a rigid structure, we learn how to shape models to fit the problem itself—moving from abstract usage to intentional design.

In short: We don't just *use* models anymore—we *design* them to think the way our problem requires.

References

https://www.tensorflow.org/guide/keras/functional_api

https://keras.io/guides/making_new_layers_and_models_via_subclassing/

<https://www.geeksforgeeks.org/deep-learning/sequential-vs-functional-api-in-keras/>

<https://machinelearningmastery.com/three-ways-to-build-machine-learning-models-in-keras/>

Next Section:

 [12. Introduction to TensorFlow & Keras](#)